Multi-Scale Deformable Attention (MSDA)

Abstract

We implement Multi-Scale Deformable Attention (MSDA) kernels on Tenstorrent's Wormhole architecture, leveraging the TT-Metal software stack. Inspired by Deformable DETR and optimized for TT's async compute model, this implementation features parallelization over queries, custom bilinear interpolation and prefix sum in device code.

Introduction

Deformable Attention aims to reduce the cost of traditional dense attention by attending to a sparse set of key-value pairs based on learned offsets. Instead of attending to all keys, each query attends to a limited number of spatial sampling points across multi-scale feature maps.

This work targets Tenstorrent's **Wormhole** accelerator using the **TT-Metal** programming model. We build a custom fused operation to:

- Read and interpolate spatially aligned values across multiple levels.
- Weight them using learned attention weights.
- Accumulate sampled values using overlapped intermediate CB.

Background

Deformable Attention: A Theoretical Overview

Deformable Attention is a sparse variant of traditional self-attention mechanisms designed to reduce the quadratic complexity in vision tasks involving high-resolution inputs. Unlike standard attention, which computes attention scores over all keys, deformable attention restricts computation to a small number of sampled key positions for each query. These sampling positions are learned as **offsets** and vary across different queries, heads, and feature levels.

This mechanism was first introduced in **Deformable DETR**, where it was used to efficiently model long-range dependencies across multi-scale features in object detection.

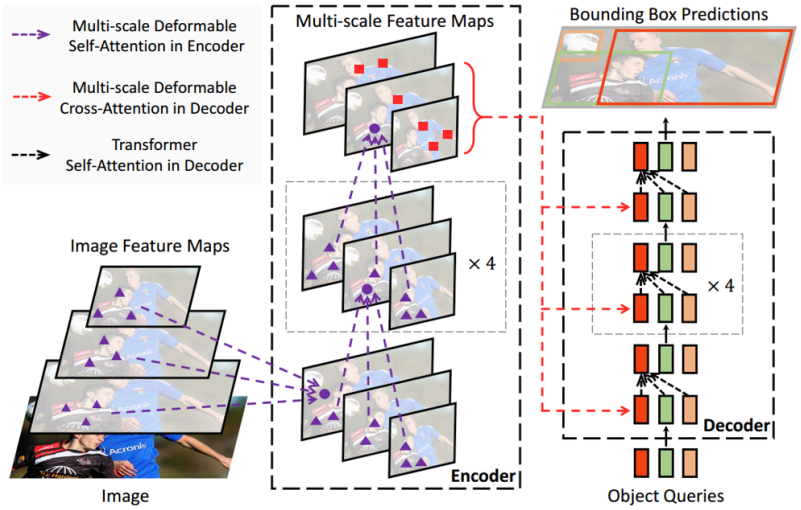


Illustration of the proposed Deformable DETR object detector.

Core Components

The operation involves four key tensors:

- `value` : Multi-scale feature maps, shaped `[B, num_keys, num_heads, embed_dims]` .
- `sampling_locations` : Normalized offsets per query, shaped `[B, num_queries, num_heads, num_levels, num_points, 2]` .
- `attention_weights` : Scalar weights per sampling location, shaped `[B, num_queries, num_heads, num_levels, num_points]` .
- `spatial_shapes` : Spatial resolution of each feature level, shaped `[num_levels, 2]` .

Computation Pipeline

1. **Offset Projection**: For each query, `sampling_locations` specify the spatial coordinates (fractional) where values should be fetched from.
2. **Bilinear Interpolation**: Values are sampled from the feature maps via bilinear interpolation at the specified coordinates.
3. **Weighting**: Each sampled feature is multiplied by its corresponding attention weight.
4. **Aggregation**: All weighted values across points and levels are summed per head to produce the output for each query.

Implementation Details

Row-Major Interleaved Inputs, Output

This version of implementation is tightly integrated with TTNN and supports:

- `DRAM` , `INTERLEAVED` and `ROW_MAJOR_LAYOUT` layout for all tensors
- Calculate point coordinate and read sampling values using `READER` kernel (N-RISCV)
- Bilinear interpolation and prefixed sum at sampled coordinates using `WRITER` kernel (B-RISCV)

Validation & Output Spec

Input memory configuration:

- **Layout**: `ROW_MAJOR`
- **Memory layout**: `INTERLEAVED`
- **Data Type**: `BFLOAT16`

Input shapes:

Name	Original Shape	Feeding shape
`value`	`[B, num_keys, num_heads, embed_dims]`	`[B, num_keys, num_heads, embed_dims]`
`sampling_locations`	`[B, num_queries, num_heads, num_levels, num_points, 2]`	`[num_queries, B*num_heads*num_levels*num_points*2]`
`attention_weights`	`[B, num_queries, num_heads, num_levels, num_points]`	`[num_queries, B*num_heads*num_levels*num_points]`
`spatial_shapes`	`[num_level, 2]`	`[num_level * 2`

- Input `value` tensor still remains the shape with `embed_dims` last
- `sampling_locations` and `attention_weights` is transposed with queries dimension to be the inner dimension where all the batch, heads, levels and points are group as outter dimension, this help to *reduce the number of sticks* read by the reader, reduce data movement overhead.
- `spatial_shapes` tensor is flattened out to 1D tensor and load to each core at once (`[num_level, 2] → [num_level * 2]`)

Output:

- Memory configuration: `ROW_MAJOR` , `DRAM` , `INTERLEAVED`
- Shape `[B, num_queries, num_heads * embed_dims]`

Compute Flow Overview

Stage	Description

Reader	• Load `spatial_shapes` and calculate level start index • Loads `sampling_locations`, `attention_weights` • Loop through each point and calculate the sampling coordinates and value weights. • Read corresponding value sticks.
Writer	• Performs bilinear interpolation, weighting, and accumulation • Writes final results to DRAM

Parallelization is query-centric: each Tensix core processes a fixed number of queries with all heads and batches.

Pseudo code

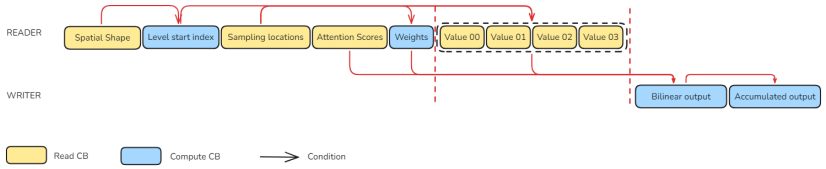
Reader

```
function READER_KERNEL(start_query_id, num_queries):
    load spatial_shapes → [Hₗ, Wₗ] for each level
    compute level_start_index from Hₗ × Wₗ for all levels

    for query_id in range(start_query_id, start_query_id + num_queries):
        # Load locations and attention weight
        load sampling_locations[query_id]
        load attention_weights[query_id]

        for batch in range(batch_size):
            for head in range(num_heads):
                for level in range(num_levels):
                    H, W = spatial_shapes[level]
                    level_offset = level_start_index[level]

                    for point in range(num_points):
                        # Get normalized (x, y) for this point
                        x_norm, y_norm = sampling_locations[batch, query_id, head, level, point]
                        x = x_norm * W - 0.5
                        y = y_norm * H - 0.5

                        if inside_bounds(x, y, H, W):
                            # Identify 4 neighbors for bilinear interpolation
                            (x0, y0), (x1, y0), (x0, y1), (x1, y1) = neighbors(x, y)

                            # Convert to flattened index per level and compute key indices
                            for each (xi, yi):
                                key_index = level_offset + yi * W + xi
                                global_index = compute_flat_index(batch, head, key_index)

                                # Read corresponding value tensor block
                                read_value_block(global_index)

                        # Compute and store bilinear weights (w1, w2, w3, w4)
                        store_to_scalar_cb(w1, w2, w3, w4)
```

Writer

```
function WRITER_KERNEL(start_query_id, num_queries):
    for query_id in range(start_query_id, start_query_id + num_queries):
        for batch in range(batch_size):
            initialize output_buffer to 0

            for head in range(num_heads):
                output_offset = head * embed_dims

                for point in range(num_levels * num_points):
                    # Wait for input values and weights from reader
                    v1, v2, v3, v4 = fetch_value_blocks()
                    w1, w2, w3, w4 = fetch_bilinear_weights()
                    attention_weight = fetch_attention_weight(point)

                    # Weighted sum of bilinear interpolation
                    result = (w1*v1 + w2*v2 + w3*v3 + w4*v4) * attention_weight

                    # Accumulate into intermediate buffer
                    intermediate_buffer += result

              # Write accumulated output
              write_output(query_id, batch, output_offset, intermediate_buffer)

```

Pipelining



The pipeline for Multi-Scale Deformable Attention (MSDA) on Tenstorrent hardware is divided between a **Reader kernel**, which prepares all necessary data, and a **Writer kernel**, which performs computation and accumulation.

Reader:

1. The reader begins by loading the spatial shape tensor and computing level start indices, which map each feature level to its offset in the flattened key tensor.
2. It then reads the sampling locations and attention weights for each query.
3. Using these, the reader computes bilinear interpolation weights based on spatial proximity and conditionally fetches four neighboring values (Value00–03) for each sampled coordinate.

Writer:

1. These values and weights are passed to the writer, which performs a bilinear interpolation, scales the result using the corresponding attention weight.
2. Accumulates the output per head.
3. Write output CB per batch to output buffer.

Memory Usage

Common Definitions

Let:

- B : Batch size
- H : Number of heads
- Q : Number of queries

- `L` : Number of levels
- `P` : Number of points
- `K` : Number of keys
- `D` : Embedding dimension
- `input_unit_size` : bytes per element in input (`bfloat16 = 2`)
- `output_unit_size` : bytes per element in output (`bfloat16 = 2`)
- `grid_unit_size` , `weight_unit_size` , `shape_unit_size` : bytes per element (`bfloat16 = 2`)
- `Float32 = 4` bytes

Per Circular Buffer Size Formulas

There are in total 11 Circular Buffers used for Deformable Attention (Row-Major, Interleaved). All Circular Buffer are currently `bfloat16` except 2 circular buffers that used for Level Start Index and Intermediate Output are `float32` for better precision.

CB Index	Name	Size Formula	Type	Size (bytes)
CB00–03	Input variants (x4)	`4 × D`	`bfloat16`	`4 × D × 2 = 8D`
CB04	Grid	`B × H × L × P × 2`	`bfloat16`	`B × H × L × P × 4`
CB05	Weight	`B × H × L × P`	`bfloat16`	`B × H × L × P × 2`
CB06	Spatial Shapes	`2 × L`	`bfloat16`	`2 × L × 2 = 4L`
CB07	Level Start Index	`L`	`float32`	`L × 4`
CB08	Scalar	`4` fixed scalars	`float32`	`4 × 4 = 16`
CB09	Intermediate Output	`D`	`float32`	`D × 4`
CB10	Final Output	`H × D`	`bfloat16`	`H × D × 2 = 2HD`

Total CB Size Formula

Full fomula to calculate total CB Size

$$\text{CB\_Size} = \underbrace{8D}_{\text{CB00–03}} + \underbrace{4 \cdot B \cdot H \cdot L \cdot P}_{\text{CB04}} + \underbrace{2 \cdot B \cdot H \cdot L \cdot P}_{\text{CB05}} + \underbrace{4L}_{\text{CB06}} + \underbrace{4L}_{\text{CB07}} + \underbrace{16}_{\text{CB08}} + \underbrace{4D}_{\text{CB09}} + \underbrace{2HD}_{\text{CB10}}$$

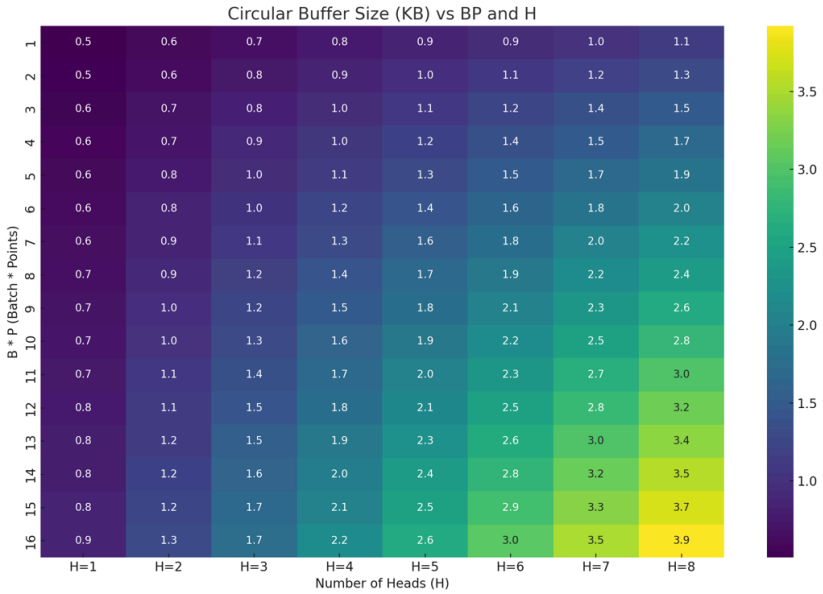
Breakdown CB Size for Row-Major Interleaved

For short:

$$\text{CB\_size (bytes)} = 12D + 2HD + 6BHPL + 8L + 16$$

Total CB Size for Row-Major Interleaved

For fixed values of `num_level` and `embed_dims` (`L = 4, D = 32`), bellow heatmap illustrates the total Circular Buffer Size (in KB) with varying of `batch_size` , `num_points` and `num_heads` (`B <= 2` , `P <= 8` and `H <= 8`). The maximum usage of Circular Buffer under this configuration is about **3.9 KB**.



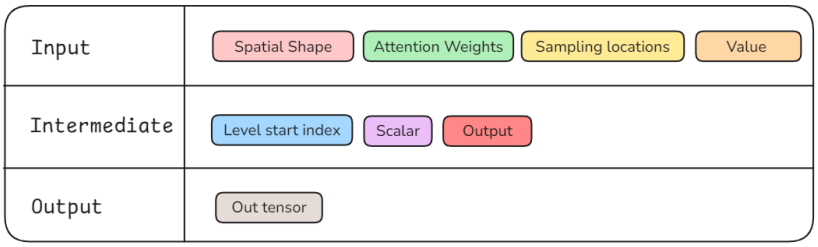
Total Size of Circular Buffer (L = 4, D = 32): Varying between B*P and H

Optimized with Compute Kernel to utilize Matrix Engine

Intention of this optimization

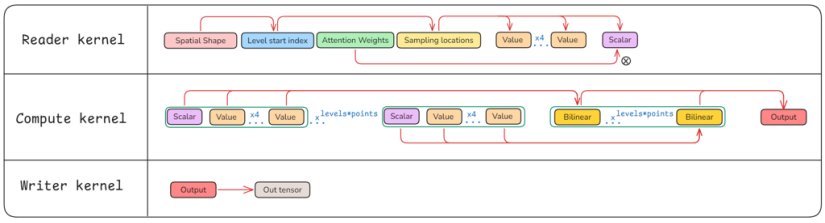
By moving the computation of Bilinear for each sampling point of `num_points*num_levels` points and following by their Summation to Matrix Engine, we can somewhat give the free for Reader and Writer kernels for doing it dedicated work and also be easier to synchronize between their kernels when having more cores to deal with the task.

Running Flow



Parameters of MSDA operation while integrate compute kernel

When having the compute kernel, we now need to reconstruct the required compile and runtime arguments for appropriate running flow.



Running flow of each kernel

The process of multiplying Bilinear Interpolation and **Attention Weight (Score)** for each sampling point now are simplifying by just the calculation of Bilinear after fusing the **Attention Weight** to the **Scalar** (Weight of each Point).

```
float weight_v = bfloat16_to_float(src2_stick[point_id]);
...
scalar[0] = float_to_bfloat16(w1*weight_v);
scalar[1] = float_to_bfloat16(w2*weight_v);
scalar[2] = float_to_bfloat16(w3*weight_v);
scalar[3] = float_to_bfloat16(w4*weight_v);
```

The Bilinear Interpolation calculation now is addressed by using a fused reduction operation which includes `tilizeA_B_reduce_init` and `unpack_tilizeA_B_block` following by `reduce_tile_math`.

After having `num_points*num_levels` Bilinear calculations, we need to sum up it into 1 point and then packing it into the output Circular Buffer for writing to the final output Tensor.

The Summation of `num_points*num_levels` is also done by the reduction operation while reduce each point in `num_points*num_levels` and still keeping it in destination registers beside setting `fp32_acc=true` in `compute_kernel_config` for the compute kernel.

For more detail, we have this configuration in Program Factory code:

```
auto reduce_op = ReduceOpMath::SUM;
auto reduce_dim = ReduceOpDim::H;
auto compute_config = ComputeConfig{
    .math_fidelity = math_fidelity,
    .fp32_dest_acc_en = fp32_dest_acc_en,
    ...
    };
```

The output of Compute kernel now is standing for 1 head of each query, so we can immediately push it into Writer kernel for writing into output Tensor after finishing it calculation.

The code for computing deformable attention at each head is:

```
    tilizeA_B_reduce_init<false, true>(in_cb_id, in_scalar_cb_id, max_tiles_per_iter, out_cb_id, 2, 4);
    pack_untilize_dst_init_short<num_output_tiles>(out_cb_id, 1, 2);

    for (uint32_t i = 0; i < nsticks_per_core; i++) {
        for (uint32_t batch_id = 0; batch_id < batch_size; batch_id++) {
            for (uint32_t head_id = 0; head_id < num_heads; head_id++) {
                tile_regs_acquire();
                for (uint32_t level_id = 0; level_id < num_levels; level_id++) {
                    for (uint32_t point_id = 0; point_id < num_points; point_id++) {
                        cb_wait_front(in_cb_id, 4);
                        cb_wait_front(in_scalar_cb_id, 1);

                        unpack_tilizeA_B_block(
                            in_cb_id,
```

```
                            in_scalar_cb_id,
                            in_ntiles_c,
                            0,
                            2
                        );

                        for (uint32_t c_i = 0; c_i < in_ntiles_c;
++c_i) {
                            reduce_tile_math(c_i, 2);
                        }

                        tile_regs_wait();

                        cb_pop_front(in_cb_id, 4);
                        cb_pop_front(in_scalar_cb_id, 1);
                    }
                }
                tile_regs_commit();
                cb_reserve_back(out_cb_id, 1);
                pack_untilize_dst<in_ntiles_c>(out_cb_id, 1, 0, 1, 2);
                tile_regs_release();
                cb_push_back(out_cb_id, 1);
            }
        }
    }

```

Performance Evaluation

Runtime Benchmark

No.	Operation	Type	Device	Num Iters	Time (ms)	Time/Iter (ms)
1	MSDA Pytorch	PyTorch	Wormhole	10	10,043.303	1004.33
2	MSDA TTNN	Composed Operation	Wormhole	10	76.977	7.698
3	MSDA TTNN (Reader + Writer)	Row-major, Interleaved	Wormhole	10	3.782	0.378
4	MSDA TTNN (Compute Kernel)	Row-major Interleaved	Wormhole	10	3.300	0.330
5	MSDA TTNN (Compute Kernel)	Row-major Interleaved	BOS-A0	10	7.215	0.722

Performance (TTNN): ~3,030 FPS on Wormhole and ~1,385 FPS on BOS-A0.

Test Case Results

Batch	Heads	Queries	Levels	Points	Keys	Dim	Similarity	Status
1	4	1	1	1	10	1	0.999984	✅ Pass
2	1	2	4	2	20	1	0.999998	✅ Pass
2	4	4	1	2	10	2	0.999991	✅ Pass
6	8	4	1	4	24	2	0.999994	✅ Pass
2	1	4	2	2	10	2	0.999996	✅ Pass
2	2	2	2	2	10	2	0.999997	✅ Pass
2	2	2	2	8	10	2	0.999996	✅ Pass
6	4	20	1	8	50	32	0.999992	✅ Pass
3	2	5	5	3	25	4	0.999995	✅ Pass
1	1	10	3	16	48	8	0.998977	✅ Pass

Pass Rate: 10/10

Debugging phases

Row-Major, Interleaved Inputs, Output

> 🎯 **Accumulation Pitfall:** During development of the operation, the accumulation step initially uses `bfloat16` intermediate output circular buffer. With exceeding large number of points and levels (e.g. `num_levels = 4`, `num_points = 16`) for wide range of input values (e.g. `[0; 1000]`), the accumulation step produces larger error yielding in output tensor. To resolve this, the Output Circular Buffer's data type is changed into `float32` to reserve the accuracy of calculation steps.

Phase	Kernel Type	Description	Status

Calculate Level Start Index	Reader	☑ Read Spatial Shape and calculate the Level Start Index	✅ Pass
Read logic of Sampling Locations and Attention Scores	Reader	☑ Read stick by stick sampling locations and attention scores ☑ Assert coresponding sticks between locations and scores within a query ☑ Assert propagate correctly through each point in one stick	✅ Pass
Calculate sampling weights	Reader	☑ Assert the sampling weight calculated from sampling locations correctly	✅ Pass
Calculate and read neighboring values	Reader	☑ Retrieve value index from location point ☑ Calculate the flatten index of 4 neighboring values and read correct value sticks	✅ Pass
Weighted Bilinear	Writer	☑ Retrieve the correct weight for sampling value ☑ Bilinear logic for 4 neighboring values given from reader	✅ Pass
Accumulation	Writer	☑ Accumulate the output stick by adding all samling output value for all points within one level	✅ Pass
Write to Output Buffer	Writer	☑ Calculate the index output stick for write output ☑ Write the corresponding output CB to correct output stick	✅ Pass

---

Future Work

- **Debug failing test cases** with low PCC.
- **Kernel optimization**: Leverage SFPU compute engine to vectorize compute process.

References

- MS-Deformable Attention (Zhu et al.)
- TT-Metal GitHub